Building AI Agents with LangGraph

Practical lab report · Advanced Artificial Intelligence / Generative AI

Reda Ghadouani · PhD student · October 2026

1. Objective and setup

The lab builds LangGraph workflows of increasing complexity: a one-node state graph, an LLM node, a two-stage analyzer → answer pipeline, conditional routing, a tool-using agent, conversational memory and a small evaluation harness. These are combined in the capstone, an **Intelligent University Assistant**. All code cells of the statement were run unchanged except the LLM connection.

Item	Value
Environment	Windows 11 · Python 3.14.6 · LangGraph 1.2.18 · LangChain 1.4.7 · Jupyter
LLM	`google/gemma-4-31b-it:free` on OpenRouter (OpenAI-compatible API), temperature 0, `max_retries=5`
Fallbacks	`poolside/laguna-s-2.1:free`, then `openrouter/free` (any free model able to serve the request)
API key	read from an environment variable, never written in the notebook
Free-tier limits	20 requests/min, 50 requests/day (reset 00:00 UTC). One full run ≈ 50 model calls

Why OpenRouter. The statement uses the OpenAI API. OpenRouter exposes the same API format, so only `base_url`, `api_key` and the model name change in the `ChatOpenAI` constructor; every other cell works as written. Free models are served by a single upstream provider shared by all free users. During the work, a call failed with `429 Provider returned error` even though the daily quota was unused: that provider was saturated. The `extra_body={"models": [...]}` fallback list fixes this: OpenRouter tries the next model. The trade-off is that a fallback model may answer some calls instead of Gemma.

2. Answers to the conceptual questions

2.1 Roles of LangChain, LangGraph and the LLM (Exercise 1)

1. **LangChain**: the toolkit of building blocks for LLM applications. It gives one common interface to many model providers, plus prompts, message types, tools (`@tool`), retrievers and output parsers. In this lab, `ChatOpenAI` works with OpenRouter just by changing `base_url`. On its own, LangChain composes these pieces into mostly linear chains.
2. **LangGraph**: the orchestration layer built on top of LangChain. It runs stateful, multi-step agents as a graph: a typed shared state, nodes, normal and conditional edges, and cycles (the agent → tools → agent loop). It adds checkpointing (memory per `thread_id`), streaming and human-in-the-loop. LangChain provides the parts; LangGraph decides the order in which they run and keeps the state between steps.
3. **An LLM**: the reasoning and generation engine. It writes the text, classifies the question and decides whether to call a tool and with which arguments. It does not run tools itself: it only emits a tool call, which a `ToolNode` executes. It is stateless, so memory has to be provided by the application. Its output is probabilistic. In this lab: `google/gemma-4-31b-it:free` on OpenRouter, with free fallback models.

2.2 Nodes, edges, START, END and state (Exercise 2)

1. **Node**: a Python function that receives the current state and returns a partial update: only the keys it changes. Example: `process_message` reads `message` and returns only `{"response": ...}`.

2. **Edge**: the control flow: which node runs after another. A normal edge is fixed (`analyzer → answer`). A conditional edge calls a routing function on the state to choose the next node (`classifier → math | programming | ai`).
3. `START`: the virtual entry point of the graph. The edge from `START` tells LangGraph which node runs first. The dictionary passed to `invoke()` becomes the initial state.
4. `END`: the virtual exit point. When the flow reaches `END`, execution stops and `invoke()` returns the final state.
5. **Why the state is passed**: nodes never call each other; the state is their only communication channel. Each node reads what earlier nodes wrote (`generate_answer` reads the `analysis` written by `analyze_question`) and adds its own fields. LangGraph merges each update: it overwrites by default, or uses a reducer such as `add_messages`, which appends. Because everything lives in one state object, it can be checkpointed (memory), inspected and debugged. In the first run the output kept `message` and added `response`.

2.3 Routing evaluation: where did the router fail, and how to make it more robust?

Where it failed: nowhere in this run. The evaluation gave **6/6 = 100 %**. The three Part VI questions and all capstone queries were also routed correctly, including "My name is Ahmed." → GENERAL.

Why that is not proof of robustness: the test set is small and every question clearly belongs to one category. The router also has weaknesses that this set does not exercise:

- It parses free text with substring checks. An answer like "programming, not mathematics" goes to `math` because math is checked first. A short label like "math" does not contain "mathematics", so it falls through to the `ai` default.
- The default branch hides errors: any unexpected output silently becomes `ai` (or `GENERAL` in the capstone).
- Mixed questions are ambiguous, for example "Write Python code to compute a derivative".
- With OpenRouter fallbacks, a different model can answer, and its label formatting can change ("**Mathematics**", "Category: math").
- Each routing decision costs an extra LLM call. In this run, classification and answering took 3–31 s per question on the free tier, largely queueing and retries.

What would make it more robust:

- **Structured output with an enum:** a Pydantic model with a `Literal[...]` category, used through `llm.with_structured_output(...)`, so only valid labels can come back. This needs a model that supports tool calling or JSON mode.
- **Validation and mapping:** map synonyms ("math" → mathematics) and send invalid labels to an explicit fallback, or ask a clarifying question instead of silently picking a default.
- **Few-shot examples** in the prompt, including borderline cases, with temperature 0 (already set).
- **A cheap first pass** (keywords or embeddings) for obvious cases, with the LLM only for the uncertain ones.
- **A larger labelled test set** with ambiguous and multi-topic questions, reporting a confusion matrix, latency and the model that actually answered.

3. Results of the experiments

Part	What was tested	Observed result
II · Ex. 2	One-node graph, shared state	`{'message': 'Hello LangGraph!', 'response': 'Agent received: Hello LangGraph!'}`, identical to the expected result
III · Ex. 3–4	LLM connection, LLM node	"Hello!"; a correct three-sentence definition of AI
IV · Ex. 5	Analyzer → answer generator	Long, structured and correct answer on reinforcement learning (agent, environment, reward, policy, Q-learning, PPO …)
V–VI · Ex. 6–7	Router + 3 specialised agents	3/3 routed correctly (mathematics, programming, artificial_intelligence); d/dx $x^2 = 2x$
VII · Ex. 8	Tools	`calculator_unsafe` and `calculator` → 110; `__import__('os')` → "Invalid expression" in the safe version
VIII	Tool-using agent	"125 * 38" → tool call `calculator(125 * 38)` → 4750 → "125 × 38 = 4,750"; "capital of Morocco" → no tool → Rabat
IX · Ex. 9	Memory	Without a checkpointer: "I don't have that information". With `MemorySaver`, same thread: "Your name is Ahmed." Other thread: name unknown
Evaluation	Routing accuracy, latency	**6/6 = 100 %**; 3.2–31.4 s per question (mean 19.6 s), including the specialised answer
X · Capstone	6 demo queries	COURSES, PROGRAMMING, MATHEMATICS (4,767), ADMINISTRATION, GENERAL ("Friday, 30 October 2026", the date-tool value), EMPTY → fallback message
X · Capstone	Memory + tool trace	"What is my name?" → "Your name is **Ahmed**."; "(15 + 27) * 3" → `calculator` → 126 → final answer

4. Capstone: Intelligent University Assistant

The question goes to a **classifier** (one LLM call) that returns one of five categories. A conditional edge sends it to the matching **specialised agent**: same LLM with the tools bound, but its own system prompt (e.g. the administration agent must not invent rules or deadlines; the mathematics agent must use the calculator). If the agent's reply contains a tool call, the shared `ToolNode` runs it and `route_after_tools` sends the result back to the **same** agent, which writes the final answer. A `MemorySaver` checkpointer stores the message list per `thread_id`.

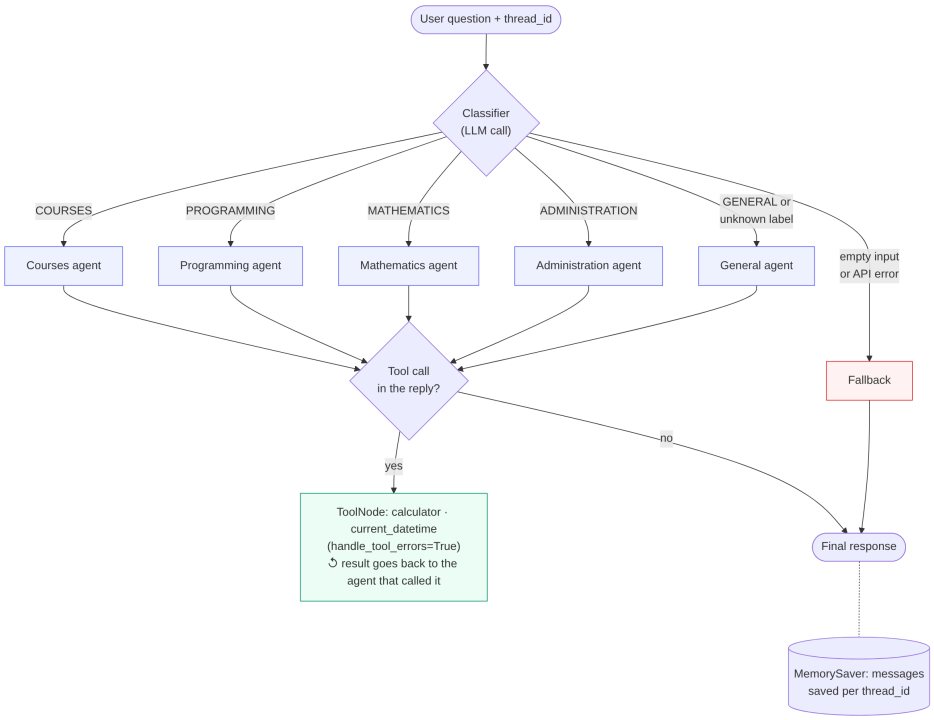


Figure 1 — Architecture of the assistant

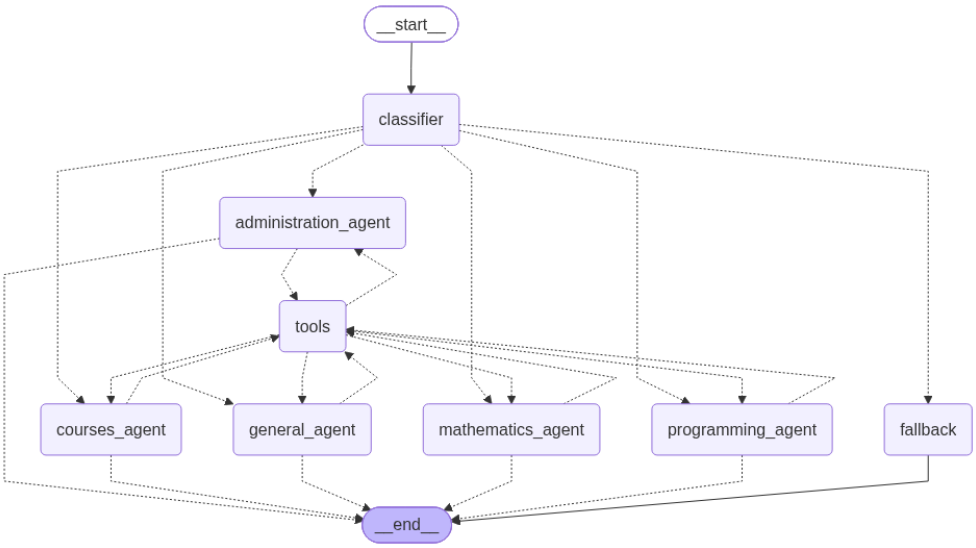


Figure 2 — Graph compiled by LangGraph (show_graph(assistant))

Error handling

Failure	Handling	Verified
Empty question	classifier returns EMPTY without calling the LLM → `fallback` message	notebook demo + test
Unknown / malformed label	mapped to GENERAL	offline test
API error in the classifier	category ERROR → `fallback` ("please try again")	offline test
API error in an agent	apology `AIMessage` with the exception type	by construction
Invalid tool call / tool exception	`ToolNode(handle_tool_errors=True)` returns the error as a ToolMessage; the agent can recover	offline test
Rate limit (429)	`max_retries=5` + OpenRouter model fallbacks	observed during the lab

5. Observations and limitations

- **Tool use is a decision of the model.** It worked as intended: arithmetic produced calculator calls, factual questions did not. The date question was answered with the exact local date ("Friday, 30 October 2026"), which the model can only get from `current_datetime`. For "125 * 38 + 17" the demo printed only the final answer (4,767, correct), so the output of that run does not show which tools were used. The separate trace run shows the full call/result/answer sequence.
- **Memory belongs to the application, not the model.** The LLM is stateless. The checkpointer re-sends the thread's history, so the context grows with every turn and each turn costs more tokens.
- **Cost of the agentic structure.** Each question costs at least two LLM calls (classifier + agent), and a third when a tool is used. On the free tier, this means 3–31 s per question and the 50 requests/day limit, which one full run of the notebook almost exhausts.
- **Output formatting.** Some answers use LaTeX (`$\boxed{126}$`, `\(x^2\)`), which a console does not render. A system prompt asking for plain text would fix this for a CLI or chat interface.
- **Reproducibility.** With fallbacks, the model that answered is not guaranteed to be Gemma. For a fair evaluation, log `response_metadata["model_name"]` for each call.
- **Grounding.** The courses and administration agents have no institution data. They correctly said so and pointed to the registrar, but a real assistant would need a retrieval tool (e.g. SQLite with course data, or a PDF reader) as suggested in the optional extensions.

6. Conclusion

LangGraph makes the control flow of an agent explicit: a typed state shared by nodes, edges and conditional edges that route on that state, cycles for tool use, and a checkpointer for memory. The capstone combines classification, five specialised agents, two tools, memory and error handling in one graph. It routed every test question correctly and used the tools when needed. The main limits seen here come from the free-tier LLM (latency, rate limits, model changes through fallbacks) and from free-text routing, which structured output with an enum would make more robust.